Deductive LLVM verification in Isabelle

Research Topics

Thomas Kas
t.s.kas@student.utwente.nl

2026-03-25

Contents

1 Introduction . . . 3
1.1 Research Goals . . . 4
2 Background . . . 5
2.1 Deductive Verification . . . 5
2.1.1 Hoare Triples . . . 5
2.1.2 Floyd's Method . . . 6
2.2 LLVM and LLVM-IR . . . 8
2.2.1 LLVM-IR Structure . . . 8
2.3 Isabelle . . . 10
3 Outline . . . 11
Bibliography . . . 21

1 Introduction

As software is used in more and more safety-critical areas, the importance of its verification grows rapidly. It is used to control modes of transportation like cars and planes, critical infrastructure like power plants and electrical grids, and many other areas. If such software fails, the consequences are great, which means verifying that such pieces of software work as intended has become crucial. The goal of this thesis is to create a verification tool for most software regardless of source language or target architecture by strategically choosing an optimal verification method and target combination.

A simple example of a C program to be verified is shown in Listing 1. It consists of only the function `mult`, which takes two parameters `a` and `b`, and should result in an integer of these inputs multiplied together. Its implementation is not quite straight-forward as it does not use any multiplication instruction, instead relying on a for-loop with repeated additions. As such, verification of even such a relatively simple program requires in-depth understanding of the semantics of C.

```c
int mult(int a, int b) {
    int result = 0;
    for (int i = 0; i < b; i++) {
        result += a;
    }
    return result;
}
```

Listing 1: C `mult` Function Implementation

This verification can be done through various different methods. One such method is model checking, where a model of the system the software acts upon is created and used to verify the software functions correctly in some or all possible states of the model. Another method is dynamic analysis, where (part of) the software is run and its results or execution states are checked against some expected outcome or allowed bounds, commonly implemented in unit or acceptance tests. In contrast to dynamic analysis, static analysis techniques can be used to identify issues without running the software. The issues it can identify usually consist of common programming mistakes and is limited in its scope, lacking the strength to verify an implementation matches the expected behavior. Finally, there is deductive verification, which mathematically verifies correctness using logical reasoning based on pre- and post-conditions. It is using this last method, deductive verification, that a new verification tool will be created in this thesis, as its formal mathematical basis allows for the most rigorous verification of these approaches.

Deductive verification can be done at various stages in compilation of software. It might be done with the source code, with fully compiled binaries, or with some intermediary between these stages. Performing verification on source code comes with the advantage that it is relatively easy to integrate the verification into the program, for example with

in-line verification instructions, but this comes at the cost of the verification being tied to the specific programming language the software was written in. Targeting a lower level like machine instructions gets around this issue and is the closest to verifying the actual execution of a program, as compilation might introduce artifacts not present in source code, but means that only a single target is verified, tying the verification to a specific architecture. In both cases, changing the language or architecture the verification is done with means porting the verification to a different target as well, introducing more overhead for such tasks.

This thesis aims to create a tool for deductive verification at a level between these two, namely LLVM-IR. This is the intermediate representation of LLVM, a compilation framework that has separate front-ends and back-ends for different source languages and target architectures. It allows language-engineers to only target a single compilation representation, LLVM-IR, while supporting many different target architectures, including x86, ARM, and RISC-V. Furthermore, it allows developers working on new hardware targets to implement a single back-end for LLVM-IR, automatically enabling support for all languages that compile to it. For our proposed verification tool this means that, once functional, it will support software written in a wide range of source languages and targeting many target architectures.

Finally, the chosen verification tool for this thesis is Isabelle/HOL. This is an interactive theorem prover that allows users to specify theorems using higher-order logic and mathematically prove them to be correct. Whenever a new theorem is created, it gives the user proof (sub-)goals that have to be proven for the theorem to be accepted. Different tools and solvers are provided to the user which aid in simplifying, breaking up, and eventually proving these goals. Furthermore, it is possible to define custom proof methods for specific tasks using the Eisbach infrastructure. Isabelle is also a functional programming language, allowing users to create their own datatypes and functions, and letting them use these in their lemmas. All of these features combined make Isabelle well-suited to defining the structure and semantics of LLVM-IR and creating verification tools that are straightforward to use on top of that.

1.1 Research Goals

With that, the goal of this thesis will be to create a deductive verification tool for partial correctness of LLVM-IR programs within Isabelle/HOL. This can be achieved through the following steps, producing a minimum viable product:

1. defining the semantics of a subset of LLVM-IR in Isabelle,
2. implementing deductive verification infrastructure that works with these semantics,
3. creating tools to efficiently and effectively verify LLVM-IR programs,
4. use these tools on some program to show its efficacy.

Once that is done and an MVP has been produced it can be extended in multiple ways, such as:

- extending the defined subset of LLVM-IR to support more of its features, instructions, and intrinsics,
- creating another tool to port LLVM-IR output into the verifier and vice versa, preferably using Isabelle to prove it can losslessly port LLVM-IR programs,
- improving the backend using advanced verification principles to make verification easier,
- extend the verifier to prove total correctness rather than only partial correctness.

2 Background

This section provides some background information on the different concepts and tools that lay at the foundation of this thesis. Specifically, Section 2.1 explains how deductive verification works and how it is done, then Section 2.2 explores how LLVM-IR is structured and how it works semantically, and finally Section 2.3 introduces the different features of Isabelle/HOL that will be used to achieve the different goals of this thesis.

2.1 Deductive Verification

Deductive verification is a method to mathematically verify the correctness of programs based in pre- and post-conditions. There exist many different deductive verifiers for different languages and toolchains, based on mostly the same principles.

2.1.1 Hoare Triples

At the core of deductive verification lie pre-conditions, commands, and post-conditions. The combination of the three, often referred to as Hoare triples, allows one to reason about programs. The pre-conditions, P, represent some conditions that have to hold for the command, c, to execute properly, while the post-conditions, Q, represent the intended result of the command.

```c
/*@
requires a * b < 2^31
requires b >= 0;
ensures \result == a * b;
*/
int mult(int a, int b);
```

Listing 2: C `mult` Function Annotated with ACSL Specifications

Consider Listing 2. It contains a specification for the function `mult`, written in ACSL, a specification language used in deductive verification of C programs using Frama-C. This specifies two pre-conditions and one post-condition for the function, which represents a command. Specifically, the pre-conditions state that the inputs `a` and `b` multiplied should not exceed the signed 32-bit integer limit and `b` should be non-negative for the function to execute properly. If this holds, then the post-condition states that the result will be the inputs `a` and `b` multiplied.

To prove that a post-condition follows from the pre-conditions and the command, the semantic meaning of the command has to be described formally. Consider again the implementation of the `mult` function in Listing 1. To prove anything about this function, the prover needs to know what exact effects the instructions have on the execution state: how values of the `int` type behave, how assigning variables behaves, how different operations such as ++ and += interact these values, and how control-flow is impacted by the `for`-loop and `return` keyword.

Once the prover knows the semantics of different instructions, it can assume the pre-conditions hold for the inputs and execute the instructions of a program symbolically, keeping track of how execution state and flow have been impacted thus far. When a return instruction is reached, the post-condition can be checked on the returned value and reached execution state. If it holds for all possible executions, then the program satisfies the specification and is verified deductively. Note that this only proves partial correctness: it might be possible for the program not to terminate, but if it does, then the post-condition holds.

2.1.2 Floyd's Method

With that, it is crucial to know the control-flow of all possible executions of a program, as it does not suffice to show correctness for only some executions. For this purpose, Floyd's method can be applied. This involves creating a control-flow graph of the program and creating specifications for all branch-points in said graph. What results is a number of instruction blocks, each annotated with some pre-conditions, and a graph containing the flow between these blocks.

Verification of programs using the annotated instruction blocks and their flow becomes straight-forward. Starting with the first block, its pre-conditions are assumed to hold, and its instructions are executed symbolically. From there, the resulting execution state has to be shown to satisfy the pre-conditions of all possible next instruction blocks. This has to be repeated for all of the annotated blocks. Finally, whenever a block ends by returning, the post-condition of the program has to be shown to hold from the pre-condition and execution of the block. When this is done, the program has been verified to be (partially) correct.

Consider Listing 3. Since the for-loop represents a branch-point in control-flow, some pre-conditions have to be specified for these instruction blocks. For this purpose, loop invariants have been added to the for-loop in the `mult` function. The pre-conditions of the function become the pre-conditions of the initial instruction block, while the loop invariants become the pre-conditions for the loop and final instruction blocks. This is visualized in Figure 1.

```c
/*@
requires a * b < 2^31;
requires b >= 0;
ensures \result == a * b;
*/
int mult(int a, int b) {
    int result = 0;
    /*@
    loop invariant 0 <= i <= b;
    loop invariant result == a * i;
    */
    for (int i = 0; i < b; i++) {
        result += a;
    }
    return result;
}
```

Listing 3: C `mult` Function Implementation Annotated with ACSL Specifications

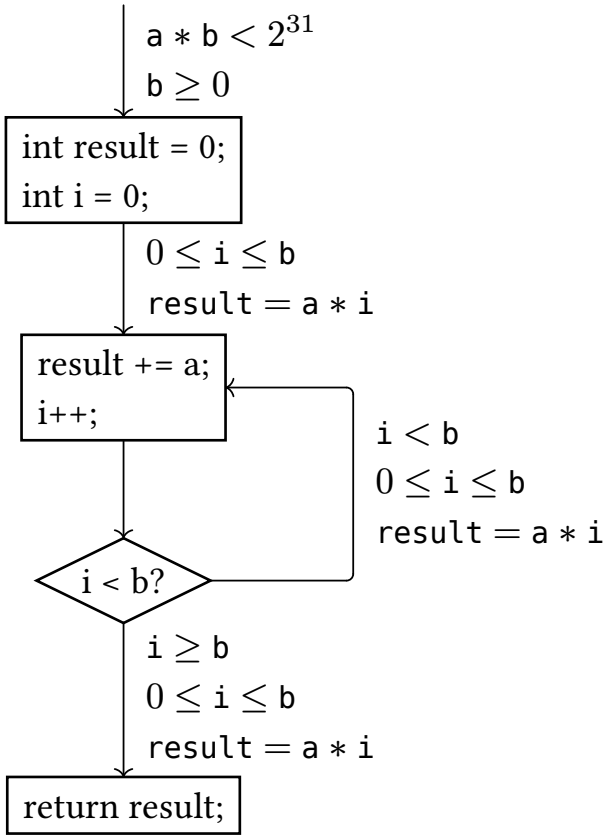

Figure 1: Annotated `mult` Control Flow Graph

From this, the goal of the deductive verification infrastructure is to:

- allow users to specify pre-conditions per block of instructions with linear control-flow and a general post-condition,

- allow for the symbolic execution of instruction blocks with some assumed conditions, obtaining the execution state,
- allow proving that the pre-conditions of all consequent instruction blocks hold for some obtained execution state, or the post-condition in case the block returned some value.

2.2 LLVM and LLVM-IR

LLVM is a toolkit for the construction of compilers, optimizers, and run-time environments. At its core is its own intermediate representation, LLVM-IR, a low-level programming language similar to assembly. It can be used to develop front-ends for any programming languages, and back-ends for any target architectures. Currently supports compiling many source languages, including C, C++, Haskell, and Rust, to a wide range of target architectures, including x86-64, ARM, and RISC-V.

2.2.1 LLVM-IR Structure

LLVM-IR is composed at the highest level of different modules, which themselves are made up of functions, global variables, and symbol table entries. Its functions consist of .....

```llvm
define dso_local noundef i32 @mult(int, int)(i32 noundef %a, i32 noundef %b) {
entry:
  %a.addr = alloca i32, align 4
  %b.addr = alloca i32, align 4
  %result = alloca i32, align 4
  %i = alloca i32, align 4
  store i32 %a, ptr %a.addr, align 4
  store i32 %b, ptr %b.addr, align 4
  store i32 0, ptr %result, align 4
  store i32 0, ptr %i, align 4
  br label %for.cond

for.cond:
  %0 = load i32, ptr %i, align 4
  %1 = load i32, ptr %b.addr, align 4
  %cmp = icmp slt i32 %0, %1
  br i1 %cmp, label %for.body, label %for.end

for.body:
  %2 = load i32, ptr %a.addr, align 4
  %3 = load i32, ptr %result, align 4
  %add = add nsw i32 %3, %2
  store i32 %add, ptr %result, align 4
  br label %for.inc

for.inc:
  %4 = load i32, ptr %i, align 4
  %inc = add nsw i32 %4, 1
  store i32 %inc, ptr %i, align 4
  br label %for.cond

for.end:
  %5 = load i32, ptr %result, align 4
  ret i32 %5
}
```

Listing 4: `mult` Function compiled to LLVM-IR

2.3 Isabelle

In order to do any kind of deductive verification, some proof assistant is needed. It has to be able to formally represent LLVM-IR and describe its exact semantics, and it needs to be able to mathematically prove theorems stated using these semantics. The interactive theorem-prover Isabelle is well-suited to this task.

Moreover, Isabelle is a functional language and allows users to create functions that can be used within proofs. Most recursive functions are automatically proven to terminate, but some require the user to prove they terminate. Pattern matching can be used in the definitions of functions. For example, see the `mult` function written as a recursive function over `b` in Listing 5. Here, `a` and `b` are natural numbers.

```isabelle
fun mult where
  "mult a 0 = 0"
| "mult a (Suc b) = a + mult a b"
```

Listing 5: Isabelle `mult` Recursive Function

Creating a theorem to prove that the `mult` function indeed multiplies the two inputs is possible by stating that `mult a b` is equal to `a * b`. This can be seen in Listing 6.

```isabelle
lemma "mult a b = a * b"
  apply (induction b)
  apply simp
  apply simp
  done
```

Listing 6: Theorem of `mult` Function Result in Isabelle

The proof works as follows:

- applying induction over `b`, to get two sub-goals:
  - the base case: `mult a 0 = a * 0`
  - the inductive step: `⋀b. mult a b = a * b ⟹ mult a (Suc b) = a * (Suc b)`
- applying the simplifier to the base case, which rewrites `mult a 0` to its definition, `0`, and rewrites `a * 0` to `0` as well, and finally proves the sub-goal `0 = 0` using reflection.
- applying the simplifier to the inductive step, which rewrites `mult a (Suc b)` to its definition `a + mult a b`, uses the assumption (which is specified for any b) to rewrite `mult a b` to `a * b` resulting in `a + a * b = a * (Suc b)`, and finally proves it correct using the internal definition of multiplication of natural numbers and reflection.

3 Outline

- intro sketch
  - why is llvm deductive verification useful?
  - what methods will be used?
    - dedver
    - isabelle
    - LLVM-IR
  - research goals
  - include example

- Introduction
  - why software verification
  - which methods will be used (1-2 sentences why)
    - dedver
    - isabelle
    - LLVM-IR
  - research goals
  - motivating example

- Background
  - Deductive verification
  - Isabelle
  - LLVM(-IR)

- Preliminary Results
  - LLVM-IR semantics in Isabelle
  - Deductive verification infrastructure
  - Tools to efficiently verify programs
  - Small working examples

-

- Introduction
  - software verification
    - why is it necessary?
    - what are some different methods
      - deductive verification
      - model checking
      - unit-testing, acceptance testing, model-based testing, etc.
  - deductive verification
    - underlying principles
      - Hoare logic (+ separation logic)
      - Floyd's method
    - why choose this for this thesis?

- LLVM
  - why was it created?
  - why choose it for this thesis?
    - nearly all modern programming languages support compiling to LLVM
    - decompiling binaries to LLVM is relatively straight-forward
- Isabelle
  - what does it do?
  - why choose it for this thesis?
- what I'll do (research questions!)
  - outline of sections of thesis
    - background
    - LLVM-IR
    - implementation

- background
  - in-depth explanation of deductive verification
  - in-depth explanation of Isabelle (only the features I use!)

- LLVM-IR
  - describe syntax
    - program made up of functions
    - functions made up of instruction blocks
    - instruction blocks made up of a label, phi nodes, instructions, and terminal instructions
    - C -> LLVM-IR example
  - describe semantics
    - function executed -> first block executed
    - instructions executed sequentially
    - terminal instruction branches to a different block, or returns a value from the function
    - phi nodes assign values to a register depending on the previous block

- LLVM-IR in Isabelle (preliminary results)
  - how is syntax implemented
    - datatypes and type-synonyms
  - how are semantics implemented
    - interpreter over the IR
    - functions are executed from the first block onwards
    - blocks are executed by sequentially executing each instruction
    - flow is handled using terminator instructions, either executing another block or returning a value
    - instructions alter the state according to the semantics defined in the language reference[1]

- ! the semantics cannot be verified to be accurate, so it is trusted: assumed to be correct !

- deductive verification of LLVM-IR (TODO)
  - flow-graph of blocks constructed
  - pre- and post-conditions for any diverging blocks
  - given the pre-condition of a block, show the post-condition of the next diverging endpoint holds
  - where to define these pre- and post-conditions
  - instructions have some effect on state - described in weakest precondition rules
  - verification condition generator takes the pre- and post-conditions, and provides goals to prove
  - only partial correctness

- related works

- planning for completion of the thesis
  - implement separation logic
  - support arrays
  - create control-flow graphs of functions
  - implement VCG for multiple blocks strung together
  - allow annotating blocks with pre- and post-conditions
    - determine diverging paths
    - instantiate phi-nodes based on previous blocks
  - create a translation tool from native LLVM-IR to IsabeLLVM
    - if time permits, create a parser for it within Isabelle
    - otherwise, a simple Rust script will do
    - perhaps also the other way around
    - define MVP
    - risk analysis

[1] https://llvm.org/docs/LangRef.html

Introduction

Say we have a C function for multiplying two integers:

```c
int mult(int a, int b) {
    int result = 0;
    for (int i = 0; i < b; i++) {
        result += a;
    }
    return result;
}
```

How would one verify that the returned value is indeed equal to a and b multiplied together?

Software Verification

One way to go about verifying this result is using a software verification tool made for the program's source language. Such tools are used to assert that specific conditions hold in the execution of a program. For the given example, a reasonable option would be to use Frama-C.[1] This is a mature platform that can be used to verify specifications written in ACSL (ANSI/ISO C Specification Language), for example:

```c
/*@
requires a * b < 2^32;
requires b >= 0;
ensures \result == a * b;
*/
int mult(int a, int b);
```

This specification contains two pre-conditions and a post-condition. Specifically, the pre-condition states that the inputs multiplied together should not overflow and b should be non-negative, and the post-condition states that, if the pre-condition held, then the result of the function is equal to the two inputs multiplied together.

However, these conditions are not enough to prove the correctness of the program. They only provide the goals for the correctness proof. In order actually prove that the post-condition follows from the pre-condition, the actual program and control-flow has to be taken into account.

For Frama-C, this involves annotating parts of the implementation where the control-flow is non-linear. In our example, this is the while-loop, which has to be annotated by a loop-invariant. These are invariants that hold both at the start of the loop and after every single execution. We might define the following loop-invariant:

```c
int mult(int a, int b) {
    int result = 0;
    /*@
    loop invariant 0 <= i <= b;
    loop invariant result == a * i;
    */
    for (int i = 0; i < b; i++) {
        result += a;
    }
    return result;
}
```

With that, Frama-C is able to verify that the result will be equal to `a*b`. How this verification works under the hood is explained in. However, what if we now want to use this function in a different programming language, for example in Rust or C#? This would involve finding a new verification framework and writing new specifications in its semantics. Is it there a way to use the same verification framework for many different languages?

There are two different, but similar, formal systems for this exact purpose. These are Hoare logic, a deductive system based on axioms and inference rules[2] described in , and Floyd's method[3].

- difficult to assess whether a program always does what it's supposed to
- mostly based on Hoare Logic , sometimes
- many existing verification tools for different languages

Note that there is a difference between the domain of the integers a and b in the code and the specification. In the code, these are 32-bit integers, which means they have limits and can overflow when they become too high, while in the specification they are mathematical integers without bounds.

Deductive Verification

Hoare Logic

Hoare logic forms the basis of a lot of program verification.

- pre-conditions define what is needed to execute successfully
- post-conditions define the expected result of successful execution
- "if P holds and we execute c, then Q holds afterwards"

Floyd's Method

However, Hoare logic is not the only basis to be used for program verification. Another method was defined by Floyd independently from Hoare. Rather than defining an algebra like Hoare, Floyd's method views programs as flowcharts where each vertex represents

a command being executed. Verification conditions are defined as $V_c(P; Q)$ which assert that if P holds and command c is executed, then Q will hold afterwards.[3]

Separation Logic

- [4]
- Extension with better reasoning for programs with pointers/dynamic memory allocation
- Axiomatic specification of memory operations [5]

Existing Verifiers

Wow

LLVM

- intermediate language for compilation
- supports many source languages as it's relatively simple to compile to
- many target architectures
- built-in optimizations
- example of LLVM program
- "How would we prove that this program is correct?"

Isabelle

- interactive theorem prover: allows user to specify theorems and prove them correct
- provides user with goals to be proven and tools to prove those goals
- allows user to create functions and datatypes
- theorems can incorporate these functions and datatypes
- example of Isabelle datatype, function, and proof

  Existing imperative program verifiers

- Framework for VCGs using theorem provers [6]
- Exporting from Isabelle to LLVM[7]
- Verifying x86 binaries[8]

  Deductive verifiers

There exist many deductive verifiers for different purposes, based on different principles. Some are based on separation logic, while others use verification condition generators.

Ones based on VCGs often use a frontend/backend architecture, where the frontend translates a verification problem into some intermediate language used in verification, and the backend extracts proof obligations and proves them.[9] With this architecture, it becomes easier to support new languages for verification, as only a new frontend needs to be created rather than reimplementing all verification infrastructure. This architecture aligns with

that of LLVM-based compilers, which also use a separate frontend (translating the source language to the LLVM-IR) and backend (compiling the LLVM-IR to the target architecture).

This architecture is brought to separation-logic based verifiers by Viper. It is a verification infrastructure, consisting of an intermediate language and two internal verifiers. Its aim is to bring the frontend/backend architecture to separation logic based verifiers. It supports many different languages through separate frontends, including Rust, Java, and C.[9] An example of such a front-end is VerCors. This is a verifier aimed at concurrent programs written in languages such as Java, OpenCL, and OpenMP.[10] It has a prototype implementation to support verification of LLVM-IR programs called VCLLVM, which is not yet production-ready.[11] Its purpose is similar to that of this project: create a verifier for LLVM so that all languages compiling to LLVM are immediately supported.

Another separation-logic based verifier is VeriFast, aimed at verifying single- and multi-threaded C and Java programs.[12]

Dafny has a similar approach with a key difference: instead of compiling programs to Dafny from their source language, programmers instead create programs in the Dafny language, verify them there, and then compile them from Dafny to their preferred language.[13]

- RESOLVE[14]
- Whiley[15]
- Frama-C[1]
- KIV[16]
- OpenJML[17]

Preliminary results

Work has already begun defining the semantics of LLVM in Isabelle and creating a verification condition generator based on Floyd's assertion method using weakest precondition mechanics.

LLVM semantics

Part of the LLVM-IR's AST has been defined as datatypes in Isabelle. With this part of the AST, functions made up of instruction blocks can be executed.

The operational semantics are defined as follows:

- Functions contain multiple labeled instruction blocks and one unlabeled block to be executed first.
- Instruction blocks have a list of regular instructions to be executed in order and one terminator instruction that impacts execution flow.
- Instructions are executed according to their specification in the LLVM Language Reference Manual.[2]

[2] https://llvm.org/docs/LangRef.html

Execution of any single instruction takes in some state and produces a state. This state is defined as follows:

- It is a triple of single static assignment (SSA) values, a stack, and a heap.
- The stack and heap have the same underlying memory definition: a list of values. They are addressable using indices in these lists.
- SSA values are a mapping from a name to a value. Although LLVM only allows assigning them once per function, this only applies to their static definition, not their value in execution. As such, no such limitation is posed here. This means the verifier works on a superset of LLVM.

The regular instructions currently (partly) implemented are:

- alloca - allocates a new address in the stack, and keeps track of the address with some SSA name.
- store - stores a value at some address in the stack or heap.
- load - loads a value from some address in the stack or heap and keep track of it with an SSA name.
- icmp - compares two values (32/64 bit signed/unsigned integers) and keep track of the result as a single bit (boolean) with an SSA name.
- add - adds two values (32/64 bit signed/unsigned integers) and keep track of the result with an SSA name (another 32/64 bit integer or a poison value if an overflow occurred).
- phi - stores an SSA value from a list of possible values depending on the instruction block that lead to this block.

The terminator instructions implemented are:

- br - branch to a different instruction given its label (could always be the same label, or switch based on a boolean SSA value).
- ret - return from the function with some value.

With this implementation, basic programs consisting of these instructions can already be executed. For example, take the following C program:

```c
int main() {
    int y = 1;
    int x = y?1:0;
    return x;
}
```

This might produce the following LLVM-IR code:

```llvm
define dso_local i32 @main() #0 {
  %1 = alloca i32, align 4
  %2 = alloca i32, align 4
  %3 = alloca i32, align 4
```

```
  %4 = alloca i32, align 4
  store i32 0, ptr %1, align 4
  store i32 1, ptr %2, align 4
  %5 = load i32, ptr %2, align 4
  %6 = icmp ne i32 %5, 0
  br i1 %6, label %7, label %9

7:
  store i32 1, ptr %4, align 4
  %8 = load i32, ptr %4, align 4
  br label %10

9:
  br label %10

10:
  %11 = phi i32 [ %8, %9 ], [ 0, %9 ]
  store i32 %11, ptr %3, align 4
  %12 = load i32, ptr %3, align 4
  ret i32 %12
}
```

Encoded into the current AST representation, this becomes:

```isabelle
definition pmain :: "llvm_instruction_block" where
  "pmain = ([
    alloca ''1'' i32 (Some 4),
    alloca ''2'' i32 (Some 4),
    alloca ''3'' i32 (Some 4),
    alloca ''4'' i32 (Some 4),
    store i32 (val (vi32 0)) (ssa_val ''1'') (Some 4),
    store i32 (val (vi32 1)) (ssa_val ''2'') (Some 4),
    load ''5'' i32 (ssa_val ''2'') (Some 4),
    icmp ''6'' False comp_ne i32 (ssa_val ''5'') (val (vi32 0))],
    br_i1 (ssa_val ''6'') ''7'' ''9''
  )"

definition p7 :: "llvm_instruction_block" where
  "p7 = ([
    store i32 (val (vi32 1)) (ssa_val ''4'') (Some 4),
```

```
    load ''8'' i32 (ssa_val ''4'') (Some 4)],
    br_label ''10''
  )"

definition p9 :: "llvm_instruction_block" where
  "p9 = ([],
    br_label ''10''
  )"

definition p10 :: "llvm_instruction_block" where
  "p10 = ([
    phi ''11'' i32 [(''7'', ssa_val ''8''), (''9'', val (vi32 0))],
    store i32 (ssa_val ''11'') (ssa_val ''3'') (Some 4),
    load ''12'' i32 (ssa_val ''3'') (Some 4)],
    ret i32 (ssa_val ''12'')
  )"

definition phi_main :: "llvm_function" where
  "phi_main = func (func_def ''main'' i32) pmain [(''7'', p7), (''9'', p9), (''10'', p10)]"
```

This can be executed as follows, which gives the proper return value:

```isabelle
value "execute_function empty_state phi_main"

= "ok (Some (vi32 1))" :: "llvm_value option result"
```

Weakest precondition mechanics

- Intro rules done for abstractions, some instructions

Rough planning

- Better LLVM semantics
- Finish intro rules for instructions
- Support verification condition generation at the level of blocks
- Add method of specifying pre- and post-conditions for code blocks
- Create flowchart view of code blocks
- Match up post-condition of one block with pre-condition of subsequent block for correctness proofs
- Support more LLVM instructions
- Support arrays
- Import/export from/to direct LLVM code

Bibliography

[1] A. Blanchard, F. Loulergue, and N. Kosmatov, "Towards Full Proof Automation in Frama-C Using Auto-active Verification," in *NASA Formal Methods*, Springer International Publishing, 2019, pp. 88–105. doi: 10.1007/978-3-030-20652-9_6.

[2] C. A. R. Hoare, "An axiomatic basis for computer programming," *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, Oct. 1969, doi: 10.1145/363235.363259.

[3] R. W. Floyd, "Assigning Meanings to Programs," in *Program Verification*, Springer Netherlands, 1993, pp. 65–81. doi: 10.1007/978-94-011-1793-7_4.

[4] J. Reynolds, "Separation logic: a logic for shared mutable data structures," in *Proceedings 17th Annual IEEE Symposium on Logic in Computer Science*, in LICS-02. IEEE Comput. Soc, pp. 55–74. doi: 10.1109/lics.2002.1029817.

[5] W. Mansky, D. Garbuzov, and S. Zdancewic, "An Axiomatic Specification for Sequential Memory Models," in *Computer Aided Verification*, Springer International Publishing, 2015, pp. 413–428. doi: 10.1007/978-3-319-21668-3_24.

[6] J. Matthews, J. S. Moore, S. Ray, and D. Vroon, "Verification Condition Generation Via Theorem Proving," in *Logic for Programming, Artificial Intelligence, and Reasoning*, Springer Berlin Heidelberg, 2006, pp. 362–376. doi: 10.1007/11916277_25.

[7] P. Lammich, "Generating Verified LLVM from Isabelle/HOL," Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, pp. 22:1–22:19. doi: 10.4230/LIPICS.ITP.2019.22.

[8] J. A. Bockenek, F. Verbeek, P. Lammich, and B. Ravindran, "Formal Verification of Memory Preservation of x86-64 Binaries," in *38th International Conference on Computer Safety, Reliability and Security*, Sept. 2019.

[9] P. Müller, M. Schwerhoff, and A. J. Summers, "Viper: A Verification Infrastructure for Permission-Based Reasoning," in *Verification, Model Checking, and Abstract Interpretation*, Springer Berlin Heidelberg, 2015, pp. 41–62. doi: 10.1007/978-3-662-49122-5_2.

[10] S. Blom, S. Darabi, M. Huisman, and W. Oortwijn, "The VerCors Tool Set: Verification of Parallel and Concurrent Software," in *Integrated Formal Methods*, Springer International Publishing, 2017, pp. 102–110. doi: 10.1007/978-3-319-66845-1_7.

[11] D. van Oorschot, M. Huisman, and Ö. Şakar, "First Steps towards Deductive Verification of LLVM IR," in *Fundamental Approaches to Software Engineering*, Springer Nature Switzerland, 2024, pp. 290–303. doi: 10.1007/978-3-031-57259-3_15.

[12] B. Jacobs, J. Smans, P. Philippaerts, F. Vogels, W. Penninckx, and F. Piessens, "VeriFast: A Powerful, Sound, Predictable, Fast Verifier for C and Java," in *NASA Formal Methods*, Springer Berlin Heidelberg, 2011, pp. 41–55. doi: 10.1007/978-3-642-20398-5_4.

[13] K. R. M. Leino, “Dafny: An Automatic Program Verifier for Functional Correctness,” in *Logic for Programming, Artificial Intelligence, and Reasoning*, Springer Berlin Heidelberg, 2010, pp. 348–370. doi: 10.1007/978-3-642-17511-4_20.

[14] M. Sitaraman and B. W. Weide, “A Synopsis of Twenty Five Years of RESOLVE PhD Research Efforts,” *ACM SIGSOFT Software Engineering Notes*, vol. 43, no. 3, p. 17, Dec. 2018, doi: 10.1145/3229783.3229794.

[15] D. J. Pearce, M. Utting, and L. Groves, “An Introduction to Software Verification with Whiley,” in *Engineering Trustworthy Software Systems*, Springer International Publishing, 2019, pp. 1–37. doi: 10.1007/978-3-030-17601-3_1.

[16] G. Ernst, J. Pfähler, G. Schellhorn, D. Haneberg, and W. Reif, “KIV: overview and VerifyThis competition,” *International Journal on Software Tools for Technology Transfer*, vol. 17, no. 6, pp. 677–694, Apr. 2014, doi: 10.1007/s10009-014-0308-3.

[17] D. R. Cok, “OpenJML: JML for Java 7 by Extending OpenJDK,” in *NASA Formal Methods*, Springer Berlin Heidelberg, 2011, pp. 472–479. doi: 10.1007/978-3-642-20398-5_35.